The Journal of Open Source Software

MatchCake: A Python Simulator for Non-Interacting Fermionic Quantum Circuits with Machine Learning Applications

Jérémie Gince [1]

1 Institut Quantique & Département de Physique, Université de Sherbrooke, Sherbrooke, QC J1K 2R1, Canada

DOI: 10.21105/joss.11033

Software

- Review
- Repository
- Archive

Editor: Owen Lockwood

Reviewers:

- @TariniHardikar
- @purva-thakre

Submitted: 20 February 2026
Published: 24 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

We introduce `MatchCake`, a Python package that simulates matchgate quantum circuits (matchcircuits). Matchgates are a parity-preserving class of nearest-neighbour unitaries whose dynamics are equivalent to those of non-interacting Majorana fermions, making matchcircuits classically simulable in polynomial time while retaining rich quantum structure. Built on PennyLane (Bergholm et al., 2022) and PyTorch (Ansel et al., 2024), `MatchCake` brings automatic differentiation to this class of circuits, enabling research on classically simulable quantum machine learning models. We demonstrate it on quantum kernel methods for classification (Gince et al., 2024) and on the simulation of physical systems.

Statement of need

Quantum circuits describe the discrete time evolution of a quantum state under a sequence of gates, and are the backbone of quantum algorithms and simulations. Since quantum hardware is still maturing, efficiently simulating and analysing such circuits on classical computers is essential, both for designing quantum algorithms and for mapping the boundary between classical and quantum computational power. In general, the state space of an N-qubit system has dimension $\mathcal{O}(2^N)$, so generic simulators scale exponentially in the number of qubits. Efficient simulation is possible only for restricted circuit families whose states or evolutions admit compact, polynomial-size representations. Matchcircuits are one such family.

A two-qubit matchgate written in the computational basis

$$M(A,W) = \begin{bmatrix} a & 0 & 0 & b \\ 0 & w & x & 0 \\ 0 & y & z & 0 \\ c & 0 & 0 & d \end{bmatrix}, \tag{1}$$

is a parity-preserving nearest-neighbour unitary gate defined by two submatrices

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, W = \begin{bmatrix} w & x \\ y & z \end{bmatrix}, \tag{2}$$

subject to the constraint $\det(A) = \det(W)$. On a linear register, such matchcircuits evolve as non-interacting Majorana fermions (Dai et al., 2015). Recent work (Projansky et al., 2025)

connects them to Clifford circuits through fermionic Gaussian operations, framing matchcircuits as a natural generalization that interpolates between stabilizer dynamics and more general fermionic evolutions. Their extensions are also practically relevant: augmenting matchcircuits with ZZ interactions enables efficient simulation of interaction-limited Fermi–Hubbard systems (Mocherla et al., 2023). More broadly, matchgate circuits have found applications spanning quantum thermodynamics (Chapman & Miyake, 2018), quantum machine learning (QML) (Gince et al., 2024; Laar, 2022), and many-body simulation.

State of the field

Simulators differ by the circuit class they handle efficiently. Stim (Gidney, 2021) targets stabilizer (Clifford) circuits with $\mathcal{O}(N^2)$ complexity, while quimb (Gray, 2018) uses tensor-network methods (Biamonte & Bergholm, 2017) such as matrix product states to simulate low-entanglement circuits at a cost of $\mathcal{O}(N\chi^3)$ in the bond dimension χ. PennyLane (Bergholm et al., 2022) is a widely used Python framework for quantum circuit simulation and machine learning, supporting many backends and hardware interfaces, but it lacks efficient native support for matchgate circuits.

Its universal state-vector simulators (SVS) handle arbitrary circuits but scale exponentially in N, and its wrappers around specialized backends such as Stim and quimb do not target matchcircuits (Valiant, 2001). Users are therefore forced onto exponentially scaling simulators, even though matchcircuits admit polynomial-time $\mathcal{O}(N^3)$ classical simulation (Bravyi, 2008; Brod, 2016; Hebenstreit et al., 2020). Unlike tensor-network methods, whose efficiency relies on limited entanglement, matchcircuits derive their tractability from an underlying free-fermionic (Gaussian) structure and so support arbitrarily large entanglement. And unlike the discrete Clifford gate set, matchgates (Jozsa & Miyake, 2008) form a continuously parametrized family of two-qubit gates corresponding to free-fermionic operations. `MatchCake` fills the remaining tooling gap with a dedicated matchcircuit simulator integrated with PennyLane and PyTorch, whose automatic differentiation supports learning-based models that exploit matchgate dynamics.

Software design

Using `MatchCake`, the user provides a matchcircuit C (a list of matchgates), a product initial state $|\psi_{\text{initial}}\rangle$, and an observable

$$\mathcal{F} = \sum_j \beta_j F_j \tag{3}$$

with real coefficients β_j, whose terms are computational-basis projectors or Pauli words,

$$F_j \in \{|y\rangle\langle y| : y \in \{0,1\}^N\} \cup \{P\}\,, \tag{4}$$

where $|y\rangle$ is a computational-basis state and P any Pauli word on the N-qubit register. `MatchCake` then computes, in time polynomial in N, the expectation value

$$\text{Output} = \langle\psi_{\text{initial}}\left|C^\dagger \mathcal{F} C\right|\psi_{\text{initial}}\rangle\,. \tag{5}$$

As shown in Figure 1, $\mathcal{F}$ is evaluated term by term. The circuit C is mapped to its transition matrix R_C (Brod, 2016), and each term is Majorana-decomposed through the Jordan–Wigner

transformation (Jordan & Wigner, 1928). When $|\psi_{\text{initial}}\rangle$ is a computational-basis state and F_j is diagonal in that basis, the per-term expectation ε_j is read from a lookup table of Wick's contractions (Brod, 2016; Wick, 1950); otherwise it is obtained from the covariance matrix of the circuit-evolved state. Both routes reduce to Pfaffian computation, and the per-term values are accumulated as $\sum_j \beta_j \varepsilon_j$.

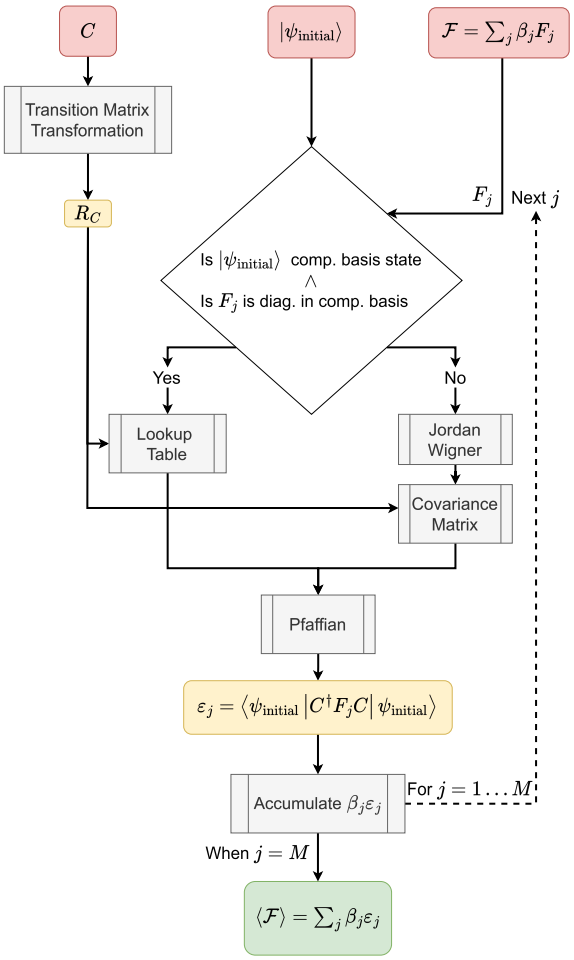


Figure 1: The `MatchCake` algorithm, which evaluates the expectation value $\langle\mathcal{F}\rangle = \sum_j \beta_j \varepsilon_j$ of the observable $\mathcal{F}$, one Pauli string F_j at a time. When the initial state $|\psi_{\text{initial}}\rangle$ is a computational-basis state and F_j is diagonal in the computational basis (Yes), the per-term expectation ε_j is read from a lookup table; otherwise (No), it is obtained from the covariance matrix of the circuit-evolved state. Both routes share a common Pfaffian evaluation.

Minimal Example - Quantum Circuit Simulation

The following example builds a matchcircuit and computes the expectation value of a computational-basis projector after applying the circuit to an initial state.

```
import matchcake as mc
import pennylane as qml
import numpy as np
from pennylane.ops.qubit.observables import BasisStateProjector

# Create a Non-Interacting Fermionic Device with 4 qubits/wires
nif_device = mc.NonInteractingFermionicDevice(wires=4)
```

```python
# Define the initial state as the all-zero computational basis state: |0000>
initial_state = np.zeros(len(nif_device.wires), dtype=int)


# Define a quantum circuit
def circuit(params, wires, initial_state):
    # Prepare the initial state
    qml.BasisState(initial_state, wires=wires)
    for i, even_wire in enumerate(wires[:-1:2]):
        idx = list(wires).index(even_wire)
        curr_wires = [wires[idx], wires[idx + 1]]
        # Apply the matchgate M(Rx(params), Rx(params))
        mc.operations.CompRxRx(params, wires=curr_wires)
        # Apply the matchgate M(Ry(params), Ry(params))
        mc.operations.CompRyRy(params, wires=curr_wires)
        # Apply the matchgate M(Rz(params), Rz(params))
        mc.operations.CompRzRz(params, wires=curr_wires)
    for i, odd_wire in enumerate(wires[1:-1:2]):
        idx = list(wires).index(odd_wire)
        mc.operations.fSWAP(wires=[wires[idx], wires[idx + 1]])
    projector: BasisStateProjector = qml.Projector(initial_state, wires=wires)
    return qml.expval(projector)


# Create a QNode
nif_qnode = qml.QNode(circuit, nif_device)
qml.draw_mpl(nif_qnode)(
    params=np.array([0.1, 0.2]),
    wires=nif_device.wires,
    initial_state=initial_state
)

# Evaluate the QNode
expval = nif_qnode(
    params=np.array([0.1, 0.2]),
    wires=nif_device.wires,
    initial_state=initial_state
)
print(f"Expectation value: {expval:.4f}")
```

Output: `Expectation value: 0.9901`

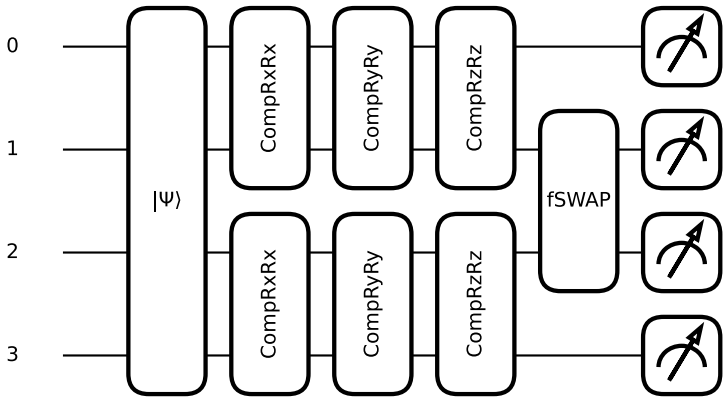


Figure 2: Quantum circuit generated by the `circuit` function.

Minimal Example - QML

The next example builds quantum kernel classifiers with the `FermionicPQCKernel` and `LinearNIFKernel` classes and evaluates them on the Iris dataset (Fisher, 1936) using 20-fold cross-validation. `MatchCake.ml` builds on `scikit-learn` (Buitinck et al., 2013; Pedregosa et al., 2011), giving a familiar interface for machine learning practitioners.

```python
import matplotlib as mpl
from matchcake.ml import CrossValidation, CrossValidationVisualizer
from matchcake.ml.kernels import FermionicPQCKernel
from matchcake.ml.kernels.linear_nif_kernel import LinearNIFKernel
from matchcake.ml.visualisation import ClassificationVisualizer
from matplotlib import pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.svm import SVC

# Load the Iris dataset
dataset = datasets.load_iris(as_frame=True)
X, y = dataset.data, dataset.target

# Define estimators using different kernels for comparison
n_qubits = 8
estimators = {
    "Fermionic PQC": Pipeline(
        [
            ("scaler", MinMaxScaler(feature_range=(0, 1))),
            # Example of kernel without alignment, which is the default behaviour
            ("kernel", FermionicPQCKernel(n_qubits=n_qubits, rotations="X,Z")),
            ("classifier", SVC(kernel="precomputed")),
        ]
    ),
    "Fermionic Linear": Pipeline(
        [
```

```
            ("scaler", MinMaxScaler(feature_range=(0, 1))),
            # Example of kernel with alignment, which improves the performance
            ("kernel", LinearNIFKernel(n_qubits=n_qubits, alignment=True)),
            ("classifier", SVC(kernel="precomputed")),
        ]
    ),
}

# Run cross-validation
cvo = CrossValidation(estimators, X, y).run()

# Create subplots for visualization
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))
# Visualize the classification boundaries for a selected model and fold
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
viz = ClassificationVisualizer(x=X, n_pts=25_000)
viz.plot_2d_decision_boundaries(
    model=estimators["Fermionic Linear"].fit(x_train, y_train),
    title="",
    legend_labels=dataset.target_names,
    y=y,
    fig=fig,
    ax=axes[0],
    legend_loc="lower right",
)
# Visualize cross-validation results with violin plots
cv_viz = CrossValidationVisualizer(cvo)
cv_viz.plot(
    ax=axes[1],
    score_name="Accuracy",
    estimator_name_key="Kernel",
    score_name_map={"train_score": "Train", "test_score": "Test"},
    palette="pastel",
)
```

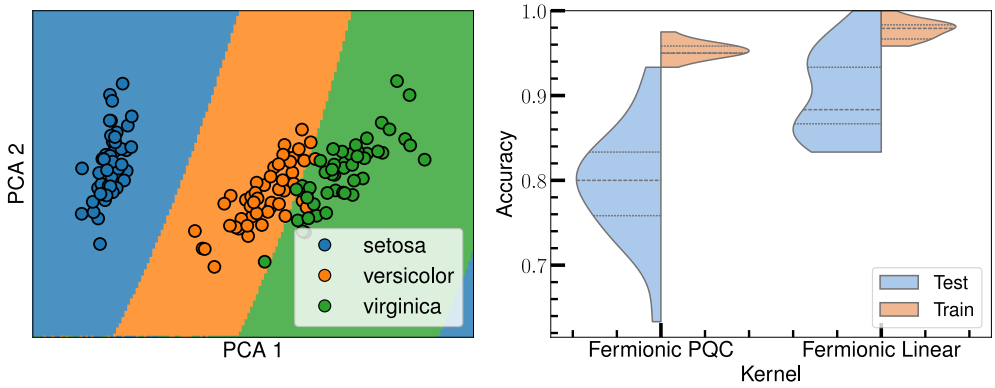


Figure 3: Output of the machine learning example, using 8-qubit kernels and 20-fold cross-validation. Left: decision boundaries for the Fermionic Linear classifier on the Iris dataset. Right: violin plots of training and testing accuracies across the cross-validation folds for the Fermionic PQC and Fermionic Linear classifiers.

Research impact statement

`MatchCake` was central to "Fermionic Machine Learning" (Gince et al., 2024), enabling the benchmarking of kernel-based free-fermionic classifiers at qubit counts infeasible with SVS.

That work underscores the need for classically simulable baselines when benchmarking QML, to test whether claimed quantum resources are actually necessary before asserting a quantum advantage. It also highlights a key strength of free-fermionic models: the provable absence of barren plateaus (Diaz et al., 2023), critical for efficient training. `MatchCake` will therefore be an invaluable tool for the quantum simulation and QML communities, supporting fundamental research and the empirical validation of quantum advantage claims.

AI usage disclosure

The author used various Claude models to improve the grammar, style, and clarity of the manuscript, and both Claude and GitHub Copilot to help draft limited parts of the source-code and documentation. Claude also helped write some tests and optimize parts of the code. No scientific content, results, or conclusions were generated by AI tools. The author made all design and architectural decisions, and has reviewed, edited, and validated all AI-assisted content and assumes full responsibility for the final version of the manuscript and the software.

Acknowledgements

The author thanks Victor Drouin-Touchette for comments on the main text and Stefanos Kourtis for his support on the topic. This research was funded by the research Chair in Quantum Computing funded by Ministère de l'Économie, de l'Innovation et de l'Énergie, the QSciTech CREATE program funded by the Natural Sciences and Engineering Research Council of Canada (NSERC), and NSERC Discovery and Alliance grants.

References

Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., … Chintala, S. (2024, April). PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. https://doi.org/10.1145/3620665.3640366

Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M. S., Alonso-Linaje, G., AkashNarayanan, B., Asadi, A., Arrazola, J. M., Azad, U., Banning, S., Blank, C., Bromley, T. R., Cordier, B. A., Ceroni, J., Delgado, A., Di Matteo, O., … Killoran, N. (2022). *PennyLane: Automatic differentiation of hybrid quantum-classical computations* (arXiv:1811.04968 [quant-ph]). arXiv. https://doi.org/10.48550/arXiv.1811.04968

Biamonte, J., & Bergholm, V. (2017). *Tensor networks in a nutshell* (arXiv:1708.00006 [quant-ph]). arXiv. https://doi.org/10.48550/arXiv.1708.00006

Bravyi, S. (2008). *Contraction of matchgate tensor networks on non-planar graphs* (arXiv:0801.2989 [quant-ph]). arXiv. https://doi.org/10.48550/arXiv.0801.2989

Brod, D. J. (2016). Efficient classical simulation of matchgate circuits with generalized inputs and measurements. *Physical Review A*, *93*(6), 062332. https://doi.org/10.1103/PhysRevA.93.062332

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API design for machine learning software: Experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.

Chapman, A., & Miyake, A. (2018). Classical simulation of quantum circuits by dynamical localization: Analytic results for Pauli-observable scrambling in time-dependent disorder. *Phys. Rev. A*, *98*, 012309. https://doi.org/10.1103/PhysRevA.98.012309

Dai, L., Kuo, W., & Chung, M.-C. (2015). Extracting entangled qubits from Majorana fermions in quantum dot chains through the measurement of parity. *Scientific Reports*, *5*(1), 11188. https://doi.org/10.1038/srep11188

Diaz, N. L., García-Martín, D., Kazi, S., Larocca, M., & Cerezo, M. (2023). *Showcasing a barren plateau theory beyond the dynamical Lie algebra* (arXiv:2310.11505 [quant-ph]). arXiv. https://doi.org/10.48550/arXiv.2310.11505

Fisher, R. A. (1936). *Iris*. UCI Machine Learning Repository. https://doi.org/10.24432/C56C76

Gidney, C. (2021). Stim: A fast stabilizer circuit simulator. *Quantum*, *5*, 497. https://doi.org/10.22331/q-2021-07-06-497

Gince, J., Pagé, J.-M., Armenta, M., Sarkar, A., & Kourtis, S. (2024). Fermionic machine learning. *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, *01*, 1672–1678. https://doi.org/10.1109/QCE60285.2024.00195

Gray, J. (2018). quimb: A Python package for quantum information and many-body calculations. *Journal of Open Source Software*, *3*(29), 819. https://doi.org/10.21105/joss.00819

Hebenstreit, M., Jozsa, R., Kraus, B., & Strelchuk, S. (2020). Computational power of matchgates with supplementary resources. *Phys. Rev. A*, *102*, 052604. https://doi.org/10.1103/PhysRevA.102.052604

Jordan, P., & Wigner, E. (1928). Über das Paulische Äquivalenzverbot. *Zeitschrift für Physik*, *47*(9), 631–651. https://doi.org/10.1007/BF01331938

Jozsa, R., & Miyake, A. (2008). Matchgates and classical simulation of quantum circuits. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, *464*(2100), 3089–3106. https://doi.org/10.1098/rspa.2008.0189

Laar, M. J. van. (2022). *Quantum aspects of variational quantum circuit-based models* [Master's thesis, Physics, University of Groningen]. https://fse.studenttheses.ub.rug.nl/28407/

Mocherla, A., Lao, L., & Browne, D. E. (2023). *Extending matchgate simulation methods to universal quantum circuits* (arXiv:2302.02654 [quant-ph]). arXiv. https://doi.org/10.48550/arXiv.2302.02654

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Projansky, A. M., Necaise, J., & Whitfield, J. D. (2025). Gaussianity and simulability of Cliffords and matchgates. *Journal of Physics A: Mathematical and Theoretical*, *58*(19), 195302. https://doi.org/10.1088/1751-8121/adcd15

Valiant, L. G. (2001). Quantum computers that can be simulated classically in polynomial time. *Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing*, 114–123. https://doi.org/10.1145/380752.380785

Wick, G. C. (1950). The evaluation of the collision matrix. *Physical Review*, *80*(2), 268–272. https://doi.org/10.1103/PhysRev.80.268